Notebook 03: **Multi-Frame Reflectivity-Style Analysis Over Motion**

1. Purpose of This Notebook

Notebook 01 established that the SemanticKITTI loading pipeline is structurally correct and numerically trustworthy. Notebook 02 then showed, on a single frame, that a simple range-aware pseudo-reflectivity proxy can become more semantically discriminative than raw intensity, even though it still remains range-dependent.

Now we move from a still photograph to a moving sequence.

The purpose of Notebook 03 is to test whether the same reflectivity-style signal remains meaningful when the robot moves through time across multiple consecutive LiDAR frames. This notebook is therefore a **temporal consistency notebook**.

We are still being honest and disciplined here:

- we are **not** claiming true calibrated reflectivity recovery,
- we are **not** training a segmentation model,
- we are **not** yet doing full sequence-scale semantic evaluation.

Instead, this notebook asks a sharper systems question:

> If a simple pseudo-reflectivity proxy looked useful on one frame, does it remain structured, stable, and interpretable across short motion windows such as 30 to 60 consecutive frames?

That is the whole dragon we are poking in this notebook.

2. Conceptual Motivation

A robotic perception system does not observe the world as isolated static frames.
It experiences the environment as a stream of measurements over time.

If a signal is genuinely useful for scene understanding, then it should not only look good in one frozen frame. It should also behave in a stable and interpretable way as:

- the platform translates through the scene,
- viewpoints shift,
- near and far surfaces change,
- scene geometry sweeps across the sensor field of view.

The project plan explicitly reserved Notebook 03 for this short temporal step, using roughly 30 to 60 consecutive frames as a feasibility-scale motion window rather than jumping too early into an oversized final system.

So the core idea here is simple:

- **Notebook 01** asked: can we trust the data pipeline?
- **Notebook 02** asked: does a range-aware signal help on one frame?
- **Notebook 03** asks: does that signal stay useful when the scene is moving relative to the robot?

That makes this notebook the bridge from single-frame signal analysis to sequence-level robotic perception.

3. Mathematical Model Used in This Notebook

3.1 Point Representation

Each LiDAR point is represented as:

$$p_i = (x_i, y_i, z_i, I_i)$$

where:

- x_i, y_i, z_i are 3D point coordinates,
- I_i is the raw recorded intensity.

3.2 Range

For each point, we compute Euclidean range:

$$R_i = \sqrt{x_i^2 + y_i^2 + z_i^2}$$

This is the same geometric quantity used in the previous notebooks.

3.3 Practical LiDAR Signal Interpretation

A standard simplified relationship for LiDAR return intensity is:

$$I_i \propto \frac{\rho_i \cos(\alpha_i)}{R_i^2}$$

where:

- ρ_i is intrinsic surface reflectivity,
- α_i is incidence angle,
- R_i is sensor range.

This matters because raw intensity is entangled with geometry and measurement conditions, not just material behavior. The uploaded paper makes this exact point and further argues that calibrated reflectivity is more interpretable than raw intensity because raw intensity is affected by range and angle of incidence.

3.4 Pseudo-Reflectivity Approximation

Since full calibration terms are unavailable in our current scope, we continue using the same heuristic proxy introduced in Notebook 02:

$$\hat{\rho}_i = I_i \cdot R_i$$

This is **not** true calibrated reflectivity.
It is only a practical, range-aware pseudo-reflectivity feature.

That honesty note stays non-negotiable.

The previous single-frame notebook already showed that $I \cdot R$ behaved more reasonably than stronger compensation such as $I \cdot R^2$, which appeared to overcompensate.

4. Procedure of This Notebook

This notebook will follow the same stepwise discipline as the earlier notebooks.

Step A: Select a short consecutive frame window

We will choose a contiguous motion segment from one SemanticKITTI sequence, typically in the range of 30 to 60 frames, consistent with the original project plan.

Step B: Load each frame and recover the raw channels

For every selected frame, we will load:

- point coordinates (x, y, z),
- raw intensity I,
- range R,
- and optionally semantic labels when needed later for interpretation.

Step C: Compute the pseudo-reflectivity proxy

For every frame, we will compute:

$$\hat{\rho} = I \cdot R$$

using the same definition as Notebook 02 so the comparison remains clean.

Step D: Build frame-wise summaries

For each frame, we will compute compact descriptive quantities such as:

- mean and standard deviation of intensity,
- mean and standard deviation of pseudo-reflectivity,
- quantiles,
- range dependence indicators,
- and possibly simple class-aware summaries if labels are attached later.

Step E: Visualize motion over time

We will generate frame-by-frame visual outputs such as:

- bird's-eye-view plots colored by raw intensity,
- bird's-eye-view plots colored by pseudo-reflectivity,
- temporal signal summaries across frame index,
- and possibly a lightweight animation or sequence of saved figures.

Step F: Compare temporal behavior

We will check whether the pseudo-reflectivity signal:

- stays visually structured across time,
- avoids collapsing into unstable noise,
- highlights scene regions consistently as the robot moves,
- and remains useful enough to justify stronger later analysis.

5. What We Are Trying to Achieve

This notebook is trying to achieve something very specific and very defensible:

Goal 1: Go beyond the single-frame story

A one-frame result is useful, but it can still be a lucky snapshot.
A short sequence result is stronger because it shows the signal survives motion.

Goal 2: Test temporal consistency

We want to know whether pseudo-reflectivity behaves like a fragile gimmick or like a reusable perception cue.

Goal 3: Establish the basis for later semantic analysis through motion

The project plan explicitly frames the later stages as semantic interpretation across motion and longer sequences, not as premature claims of novel label generation.

So Notebook 03 is where the project starts acting less like a static notebook experiment and more like the early skeleton of a robotic perception pipeline.

6. Expected Outputs

By the end of this notebook, we expect to have:

- a clean contiguous multi-frame sequence loaded correctly,
- raw intensity and pseudo-reflectivity computed for all selected frames,
- temporal plots showing how the signals evolve over frame index,
- visual progression of the scene under motion,

- evidence about whether the reflectivity-style feature remains stable and interpretable across time.

In plain terms, this notebook should tell us:

> Does the pseudo-reflectivity signal still look like a meaningful scene-understanding cue once the robot starts moving?

7. Expected Outcome

If everything behaves well, the final takeaway should be something like this:

> The pseudo-reflectivity proxy is not true calibrated reflectivity, but it remains structured and interpretable across short motion windows, making it a plausible complementary feature for reflectivity-aware robotic scene understanding.

That is exactly the kind of result we want here:

- not fake-grand,
- not overclaimed,
- but strong enough to justify the next stage.

8. Final Note

Notebook 01 was the pipeline handshake.
Notebook 02 was the single-frame interrogation.
Notebook 03 is the moving-scene stress test.

Still no segmentation hero cape.
Still no magical physics redemption arc.
Just a robot, a short sequence, and a serious check of whether this reflectivity-style signal survives contact with motion.

Step 1: Locate the sequence and define the short motion window

We begin by reusing the validated SemanticKITTI path from Notebook 01 and the short multi-frame scope from the project plan.

In this step, we are **not** loading point clouds yet. We only verify that the sequence structure exists, count available frames, and define a contiguous frame window that Notebook 03 will analyze through motion.

```
from pathlib import Path

# Notebook is assumed to run from notebooks/
DATASET_ROOT =
Path("../data/semantickitti_subset/dataset/sequences/00")
VELODYNE_DIR = DATASET_ROOT / "velodyne"
LABELS_DIR = DATASET_ROOT / "labels"
```

```python
expected_items = ["velodyne", "labels", "calib.txt", "poses.txt",
"times.txt"]

print("Dataset root:", DATASET_ROOT.resolve())
print("Exists:", DATASET_ROOT.exists())
print()

for item in expected_items:
    item_path = DATASET_ROOT / item
    print(f"{item:10} -> exists: {item_path.exists()} | path:
{item_path}")

print()

if VELODYNE_DIR.exists():
    frame_ids = sorted(p.stem for p in VELODYNE_DIR.glob("*.bin"))
    print("Total .bin frames found:", len(frame_ids))
    print("First 10 frame IDs:", frame_ids[:10])
    print("Last 10 frame IDs :", frame_ids[-10:])
else:
    frame_ids = []
    print("Velodyne directory not found.")

print()

# Short motion window for Notebook 03
START_IDX = 0
NUM_FRAMES = 30

selected_frame_ids = frame_ids[START_IDX:START_IDX + NUM_FRAMES]

print("Selected motion window")
print(" - start index :", START_IDX)
print(" - num frames  :", NUM_FRAMES)
print(" - first frame :", selected_frame_ids[0] if selected_frame_ids
else None)
print(" - last frame  :", selected_frame_ids[-1] if selected_frame_ids
else None)
print(" - valid window:", len(selected_frame_ids) == NUM_FRAMES)

if selected_frame_ids:
    all_labels_exist = all((LABELS_DIR / f"{fid}.label").exists() for
fid in selected_frame_ids)
    print(" - all matching label files exist:", all_labels_exist)
```

```
Dataset root:
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti_subset/
dataset/sequences/00
Exists: True
```

```
velodyne   -> exists: True | path:
../data/semantickitti_subset/dataset/sequences/00/velodyne
labels     -> exists: True | path:
../data/semantickitti_subset/dataset/sequences/00/labels
calib.txt  -> exists: True | path:
../data/semantickitti_subset/dataset/sequences/00/calib.txt
poses.txt  -> exists: True | path:
../data/semantickitti_subset/dataset/sequences/00/poses.txt
times.txt  -> exists: True | path:
../data/semantickitti_subset/dataset/sequences/00/times.txt

Total .bin frames found: 4541
First 10 frame IDs: ['000000', '000001', '000002', '000003', '000004',
'000005', '000006', '000007', '000008', '000009']
Last 10 frame IDs : ['004531', '004532', '004533', '004534', '004535',
'004536', '004537', '004538', '004539', '004540']

Selected motion window
 - start index : 0
 - num frames  : 30
 - first frame : 000000
 - last frame  : 000029
 - valid window: True
 - all matching label files exist: True
```

Step 2: Load the selected frames and compute the core per-point signals

Now we load the 30 selected LiDAR frames.

For each frame, we extract:

- 3D coordinates (x, y, z).
- raw intensity I.
- geometric range $R = \sqrt{x^2 + y^2 + z^2}$.
- pseudo-reflectivity $\hat{\rho} = I \cdot R$.

This step builds the basic multi-frame data structure that the rest of Notebook 03 will analyze.

```
import numpy as np

def load_semantickitti_bin(bin_path):
    scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
    xyz = scan[:, :3]
    intensity = scan[:, 3]
    return xyz, intensity

multi_frame_data = []
```

```python
for fid in selected_frame_ids:
    bin_path = VELODYNE_DIR / f"{fid}.bin"

    xyz, intensity = load_semantickitti_bin(bin_path)
    ranges = np.linalg.norm(xyz, axis=1)
    pseudo_reflectivity = intensity * ranges

    frame_dict = {
        "frame_id": fid,
        "xyz": xyz,
        "intensity": intensity,
        "ranges": ranges,
        "pseudo_reflectivity": pseudo_reflectivity,
        "num_points": xyz.shape[0],
    }
    multi_frame_data.append(frame_dict)

print("Frames loaded:", len(multi_frame_data))
print()

first_frame = multi_frame_data[0]
last_frame = multi_frame_data[-1]

print("First frame summary")
print(" - frame_id              :", first_frame["frame_id"])
print(" - xyz shape             :", first_frame["xyz"].shape)
print(" - intensity shape       :", first_frame["intensity"].shape)
print(" - ranges shape          :", first_frame["ranges"].shape)
print(" - pseudo-reflectivity   :",
first_frame["pseudo_reflectivity"].shape)
print(" - num_points            :", first_frame["num_points"])
print()

print("Last frame summary")
print(" - frame_id              :", last_frame["frame_id"])
print(" - xyz shape             :", last_frame["xyz"].shape)
print(" - intensity shape       :", last_frame["intensity"].shape)
print(" - ranges shape          :", last_frame["ranges"].shape)
print(" - pseudo-reflectivity   :",
last_frame["pseudo_reflectivity"].shape)
print(" - num_points            :", last_frame["num_points"])
print()

point_counts = [f["num_points"] for f in multi_frame_data]
print("Point count over selected window")
print(" - min :", np.min(point_counts))
print(" - max :", np.max(point_counts))
print(" - mean:", np.mean(point_counts))
```

```
Frames loaded: 30

First frame summary
 - frame_id                : 000000
 - xyz shape               : (124668, 3)
 - intensity shape         : (124668,)
 - ranges shape            : (124668,)
 - pseudo-reflectivity     : (124668,)
 - num_points              : 124668

Last frame summary
 - frame_id                : 000029
 - xyz shape               : (124018, 3)
 - intensity shape         : (124018,)
 - ranges shape            : (124018,)
 - pseudo-reflectivity     : (124018,)
 - num_points              : 124018

Point count over selected window
 - min : 120295
 - max : 124668
 - mean: 123242.7
```

Step 3: Build frame-wise summary statistics

Now we compress each frame into a small numerical summary.

For every frame, we compute descriptive statistics for:

- raw intensity.
- range.
- pseudo-reflectivity.

This gives us a clean temporal table that will let us inspect how the signal evolves across the 30-frame motion window.

```
import pandas as pd

def summarize_array(arr):
    return {
        "min": float(np.min(arr)),
        "p25": float(np.percentile(arr, 25)),
        "p50": float(np.percentile(arr, 50)),
        "p75": float(np.percentile(arr, 75)),
        "p95": float(np.percentile(arr, 95)),
        "mean": float(np.mean(arr)),
        "std": float(np.std(arr)),
    }

frame_summaries = []
```

```python
for frame in multi_frame_data:
    intensity_stats = summarize_array(frame["intensity"])
    range_stats = summarize_array(frame["ranges"])
    pr_stats = summarize_array(frame["pseudo_reflectivity"])

    row = {
        "frame_id": frame["frame_id"],
        "num_points": frame["num_points"],

        "intensity_min": intensity_stats["min"],
        "intensity_p25": intensity_stats["p25"],
        "intensity_p50": intensity_stats["p50"],
        "intensity_p75": intensity_stats["p75"],
        "intensity_p95": intensity_stats["p95"],
        "intensity_mean": intensity_stats["mean"],
        "intensity_std": intensity_stats["std"],

        "range_min": range_stats["min"],
        "range_p25": range_stats["p25"],
        "range_p50": range_stats["p50"],
        "range_p75": range_stats["p75"],
        "range_p95": range_stats["p95"],
        "range_mean": range_stats["mean"],
        "range_std": range_stats["std"],

        "pr_min": pr_stats["min"],
        "pr_p25": pr_stats["p25"],
        "pr_p50": pr_stats["p50"],
        "pr_p75": pr_stats["p75"],
        "pr_p95": pr_stats["p95"],
        "pr_mean": pr_stats["mean"],
        "pr_std": pr_stats["std"],
    }
    frame_summaries.append(row)

summary_df = pd.DataFrame(frame_summaries)

print("Summary table shape:", summary_df.shape)
print()
print(summary_df[[
    "frame_id",
    "num_points",
    "intensity_mean",
    "intensity_std",
    "range_mean",
    "range_std",
    "pr_mean",
    "pr_std"
]].head(10))
```

```
Summary table shape: (30, 23)

  frame_id  num_points  intensity_mean  intensity_std  range_mean
range_std  \
0   000000      124668        0.294134       0.147690   13.565046
10.570971
1   000001      124605        0.294588       0.148049   13.511380
10.627299
2   000002      124478        0.294162       0.148084   13.401234
10.591676
3   000003      124167        0.295665       0.149636   13.268131
10.503467
4   000004      123969        0.295585       0.151231   13.188173
10.503502
5   000005      123924        0.294694       0.153250   13.134429
10.483358
6   000006      123373        0.294909       0.156234   13.098121
10.506295
7   000007      122765        0.296495       0.159902   13.089967
10.547448
8   000008      122123        0.298687       0.161086   13.058884
10.536805
9   000009      121357        0.300832       0.161600   13.099307
10.632101

    pr_mean    pr_std
0  3.628692  2.940125
1  3.606743  2.945034
2  3.578735  2.920066
3  3.573944  2.920962
4  3.554410  2.910439
5  3.548236  2.921168
6  3.547228  2.920425
7  3.569736  2.951197
8  3.585915  2.963047
9  3.608539  3.002258
```

Brief observation

The frame-wise summary table was created successfully for all 30 consecutive frames.

A few early trends are already visible:

- raw intensity statistics change slightly across motion,
- average range remains fairly stable,
- pseudo-reflectivity mean and spread remain reasonably well-behaved across the selected window.

This suggests that the constructed signal is not collapsing numerically over time, which is a necessary first condition before deeper temporal analysis.

Step 4: Plot temporal signal statistics across frame index

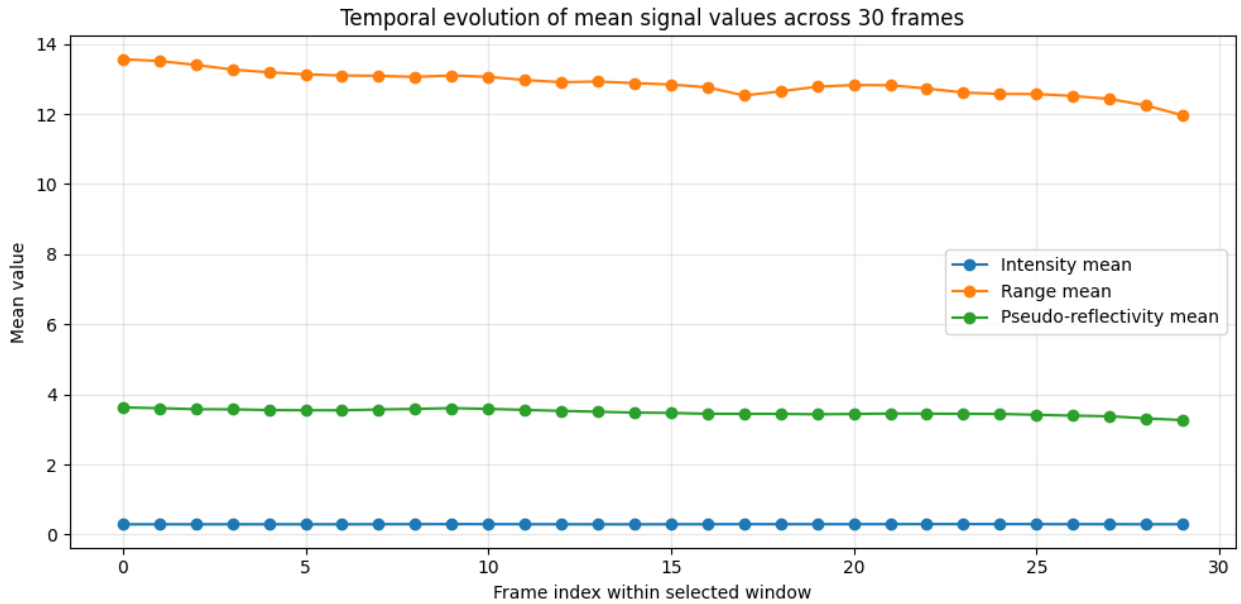
Now we visualize how the frame-wise statistics evolve over the 30-frame motion window.

We begin with the mean values of:

- raw intensity.
- range.
- pseudo-reflectivity.

This gives us the first direct temporal view of whether the constructed signal remains stable or drifts noticeably as the platform moves.

```
import matplotlib.pyplot as plt

frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["intensity_mean"], marker="o",
label="Intensity mean")
plt.plot(frame_idx, summary_df["range_mean"], marker="o", label="Range
mean")
plt.plot(frame_idx, summary_df["pr_mean"], marker="o", label="Pseudo-
reflectivity mean")

plt.xlabel("Frame index within selected window")
plt.ylabel("Mean value")
plt.title("Temporal evolution of mean signal values across 30 frames")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Brief observation

The temporal mean plot shows that all three signals remain numerically stable across the 30-frame window.

Two early patterns stand out:

- the average range gradually decreases over time,
- pseudo-reflectivity changes more gently than range and remains fairly stable overall.

Because the three signals occupy very different numeric scales, this plot is useful for trend inspection, but not ideal for direct visual comparison.

Step 5: Compare temporal trends on a normalized scale

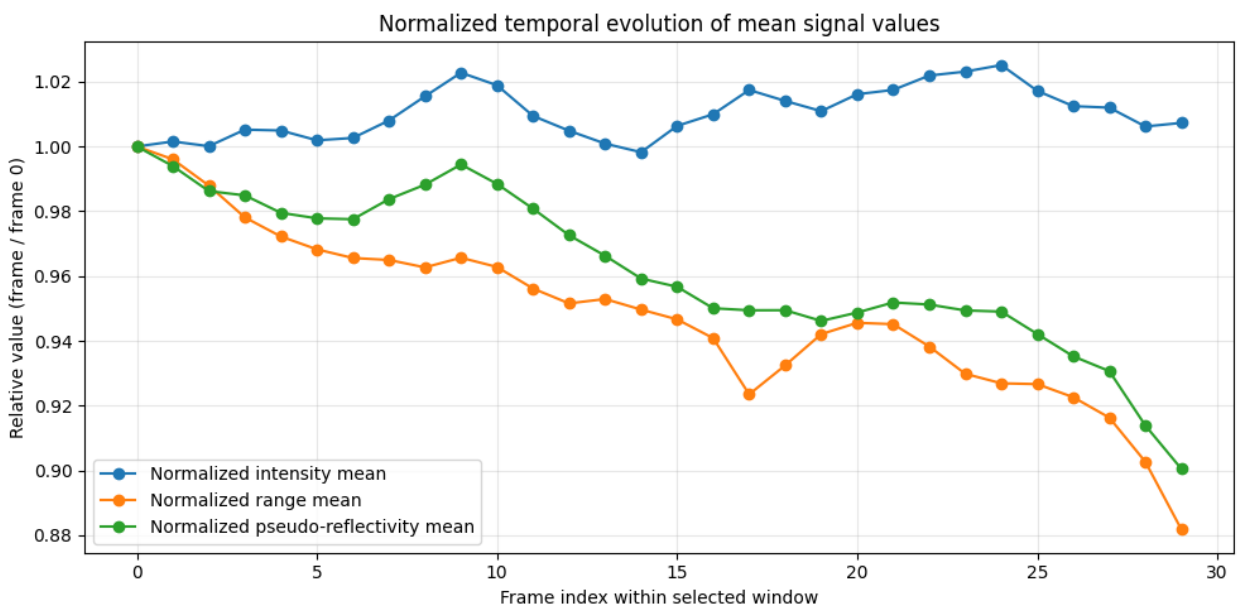
The previous plot showed that the signals are stable, but direct comparison was difficult because their numeric scales are very different.

In this step, we normalize each mean curve by its value in the first frame. This lets us compare relative temporal change across:

- raw intensity.
- range.
- pseudo-reflectivity.

```
norm_intensity_mean = summary_df["intensity_mean"] /
summary_df["intensity_mean"].iloc[0]
norm_range_mean = summary_df["range_mean"] /
summary_df["range_mean"].iloc[0]
norm_pr_mean = summary_df["pr_mean"] / summary_df["pr_mean"].iloc[0]

frame_idx = np.arange(len(summary_df))
```

```
plt.figure(figsize=(10, 5))
plt.plot(frame_idx, norm_intensity_mean, marker="o", label="Normalized
intensity mean")
plt.plot(frame_idx, norm_range_mean, marker="o", label="Normalized
range mean")
plt.plot(frame_idx, norm_pr_mean, marker="o", label="Normalized
pseudo-reflectivity mean")

plt.xlabel("Frame index within selected window")
plt.ylabel("Relative value (frame / frame 0)")
plt.title("Normalized temporal evolution of mean signal values")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Brief observation

After normalization, the temporal behavior becomes clearer.

- raw intensity mean remains relatively stable across the motion window,
- average range decreases steadily,
- pseudo-reflectivity also decreases, but in a smoother and more moderate way for much of the sequence.

This suggests that the pseudo-reflectivity signal is not unstable, but is still influenced by the changing geometric composition of the observed scene.

Step 6: Plot temporal evolution of signal spread

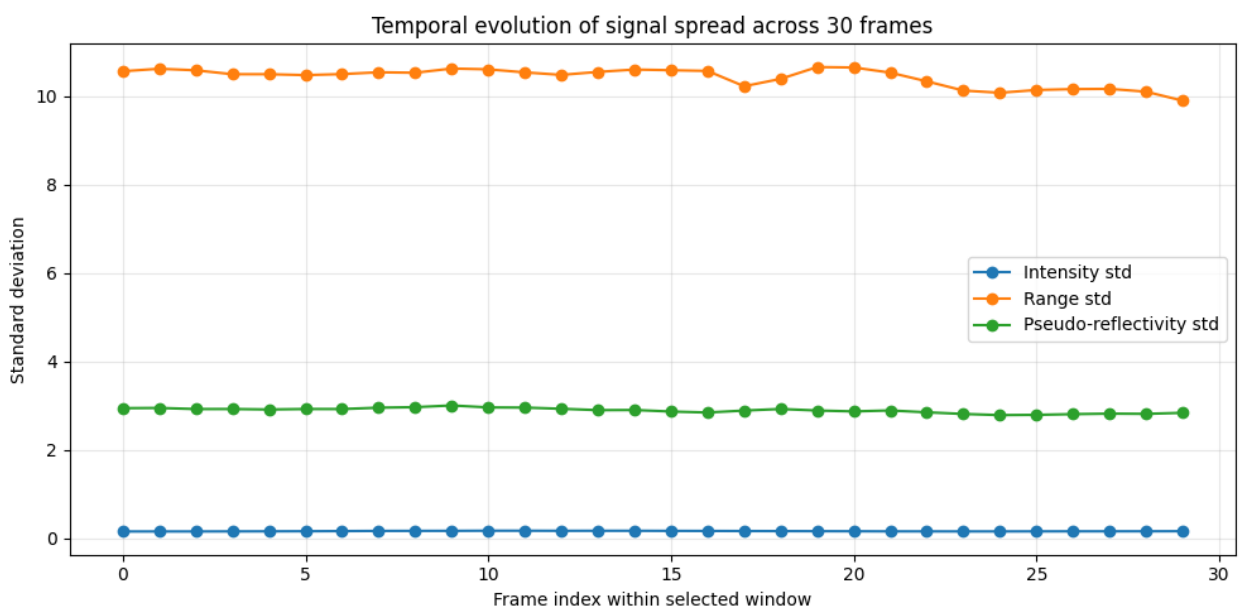
Mean values show only the center of each frame-wise distribution.

In this step, we examine the temporal evolution of the standard deviation for:

- raw intensity.
- range.
- pseudo-reflectivity.

This helps us check whether the signal distributions remain well-behaved or become more dispersed as the platform moves.

```
frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["intensity_std"], marker="o",
label="Intensity std")
plt.plot(frame_idx, summary_df["range_std"], marker="o", label="Range
std")
plt.plot(frame_idx, summary_df["pr_std"], marker="o", label="Pseudo-
reflectivity std")

plt.xlabel("Frame index within selected window")
plt.ylabel("Standard deviation")
plt.title("Temporal evolution of signal spread across 30 frames")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Brief observation

The temporal spread plot shows that all three frame-wise distributions remain well-behaved across the 30-frame window.

In particular:

- range spread is broadly stable,
- pseudo-reflectivity spread is also stable,
- raw intensity spread does not show any abrupt instability.

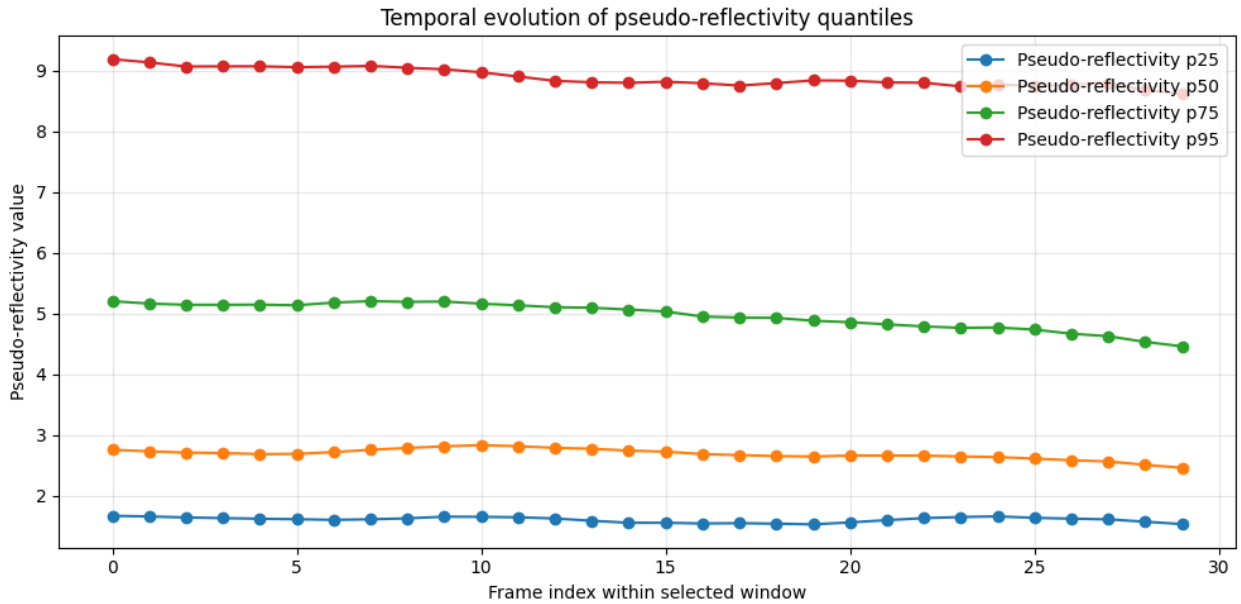
This supports the view that the constructed pseudo-reflectivity signal remains numerically controlled over short motion sequences.

Step 7: Track temporal evolution of distribution quantiles

Mean and standard deviation summarize only part of each frame-wise distribution.

In this step, we track selected quantiles over time for the pseudo-reflectivity signal, which helps us see whether the distribution shifts uniformly or changes differently across low, middle, and high-value regions.

```
frame_idx = np.arange(len(summary_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, summary_df["pr_p25"], marker="o", label="Pseudo-
reflectivity p25")
plt.plot(frame_idx, summary_df["pr_p50"], marker="o", label="Pseudo-
reflectivity p50")
plt.plot(frame_idx, summary_df["pr_p75"], marker="o", label="Pseudo-
reflectivity p75")
plt.plot(frame_idx, summary_df["pr_p95"], marker="o", label="Pseudo-
reflectivity p95")

plt.xlabel("Frame index within selected window")
plt.ylabel("Pseudo-reflectivity value")
plt.title("Temporal evolution of pseudo-reflectivity quantiles")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



The pseudo-reflectivity quantiles drift downward smoothly across the 30-frame window.

This shift is not uniform:

- **the lower and median portions of the distribution change modestly,**
- **the upper quantiles decrease more clearly over time.**

This suggests that the temporal change is not merely random noise, but a structured redistribution of stronger pseudo-reflectivity responses as the scene evolves through motion.

Step 8: Measure frame-wise range dependence across motion

So far, we have studied temporal statistics of the signal values themselves.

Now we examine a more diagnostic quantity: the correlation between signal value and range in each frame.

We compute this for:

- raw intensity vs range.
- pseudo-reflectivity vs range.

This helps us test whether the reflectivity-style proxy remains more or less range-entangled as the platform moves through the sequence.

```
framewise_corr_rows = []

for frame in multi_frame_data:
    intensity = frame["intensity"]
    ranges = frame["ranges"]
    pr = frame["pseudo_reflectivity"]
```

```
    corr_intensity_range = np.corrcoef(intensity, ranges)[0, 1]
    corr_pr_range = np.corrcoef(pr, ranges)[0, 1]

    framewise_corr_rows.append({
        "frame_id": frame["frame_id"],
        "corr_intensity_range": float(corr_intensity_range),
        "corr_pr_range": float(corr_pr_range),
    })

corr_df = pd.DataFrame(framewise_corr_rows)

print(corr_df.head(10))
print()
print("Correlation summary over 30 frames")
print(" - raw intensity vs range")
print("   mean:", corr_df["corr_intensity_range"].mean())
print("   min :", corr_df["corr_intensity_range"].min())
print("   max :", corr_df["corr_intensity_range"].max())
print()
print(" - pseudo-reflectivity vs range")
print("   mean:", corr_df["corr_pr_range"].mean())
print("   min :", corr_df["corr_pr_range"].min())
print("   max :", corr_df["corr_pr_range"].max())
```

```
  frame_id  corr_intensity_range  corr_pr_range
0   000000             -0.231389       0.398455
1   000001             -0.237417       0.390665
2   000002             -0.231691       0.392047
3   000003             -0.222040       0.402657
4   000004             -0.216446       0.399708
5   000005             -0.200680       0.404770
6   000006             -0.192225       0.391858
7   000007             -0.184621       0.383704
8   000008             -0.185355       0.375663
9   000009             -0.193318       0.368467

Correlation summary over 30 frames
 - raw intensity vs range
   mean: -0.2065390528423717
   min : -0.241687721923002
   max : -0.16775861263260272

 - pseudo-reflectivity vs range
   mean: 0.3646205896583482
   min : 0.34222221241736783
   max : 0.4047703050908886
```

Brief observation

Frame-wise correlation analysis shows that the relationship between signal and range remains consistent across the 30-frame motion window.

- raw intensity stays negatively correlated with range,
- pseudo-reflectivity stays positively correlated with range.

This means the proxy is temporally stable, but still range-dependent. In other words, it behaves as a structured range-aware signal rather than as fully geometry-invariant reflectivity.

Step 9: Visualize frame-wise range dependence across motion

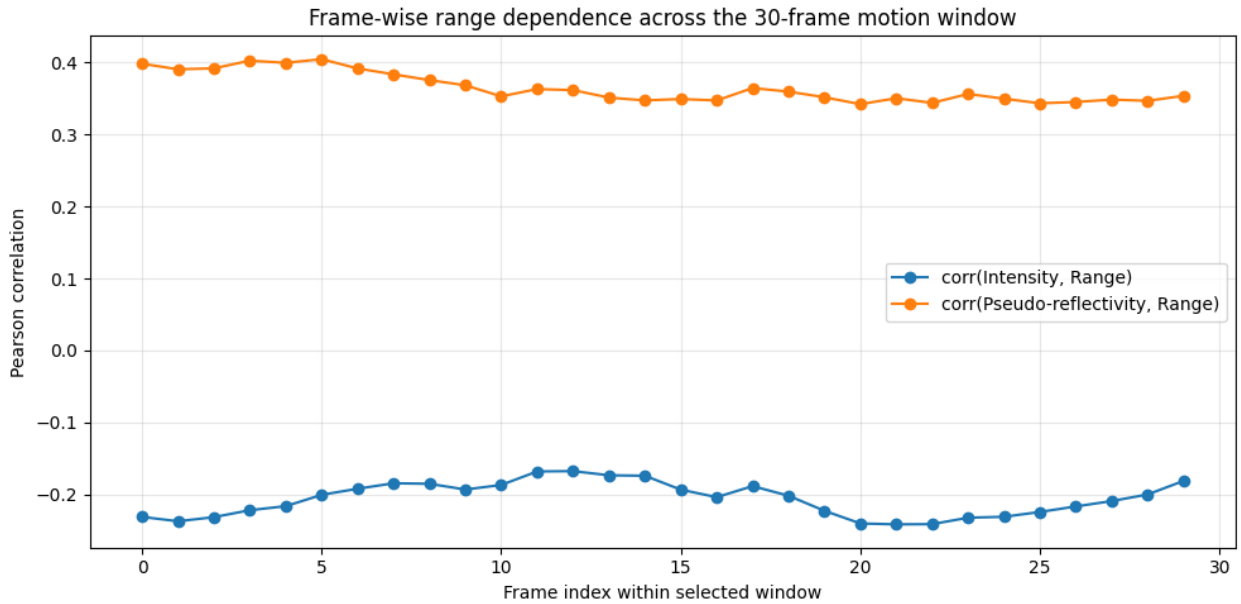
Now we plot the frame-wise signal-range correlations over time.

This lets us inspect whether the range dependence of:

- raw intensity.
- pseudo-reflectivity.

remains stable or changes meaningfully across the selected motion window.

```
frame_idx = np.arange(len(corr_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, corr_df["corr_intensity_range"], marker="o",
label="corr(Intensity, Range)")
plt.plot(frame_idx, corr_df["corr_pr_range"], marker="o",
label="corr(Pseudo-reflectivity, Range)")

plt.xlabel("Frame index within selected window")
plt.ylabel("Pearson correlation")
plt.title("Frame-wise range dependence across the 30-frame motion
window")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Brief observation

The frame-wise correlation plot shows that range dependence remains consistent across the full 30-frame motion window.

- raw intensity stays negatively correlated with range,
- pseudo-reflectivity stays positively correlated with range,
- both relationships vary only mildly over time.

This indicates that the pseudo-reflectivity proxy is temporally stable, but still retains clear geometric dependence on range.

Step 10: Recompute temporal semantic separability using the Notebook 02 metric

In this step, we use the same Fisher-style class-separability setup as Notebook 02 and apply the same class-filtering rule across frames. This lets us test, in a methodologically consistent way, whether pseudo-reflectivity remains more semantically discriminative than raw intensity over the 30-frame motion window.

```
def load_semantickitti_labels(label_path):
    raw = np.fromfile(label_path, dtype=np.uint32)
    semantic = raw & 0xFFFF
    return semantic

def get_valid_classes_notebook02_style(labels,
min_points_per_class=500, ignored_classes=(0, 1, 255)):
    valid_classes = []
    unique_classes, counts = np.unique(labels, return_counts=True)
```

```
    for cls, count in zip(unique_classes, counts):
        if cls in ignored_classes:
            continue
        if count >= min_points_per_class:
            valid_classes.append(int(cls))

    return sorted(valid_classes)

def fisher_separability(values, labels, valid_classes):
    class_means = []
    class_vars = []
    class_ns = []

    for cls in valid_classes:
        cls_values = values[labels == cls]
        class_means.append(np.mean(cls_values))
        class_vars.append(np.var(cls_values))
        class_ns.append(len(cls_values))

    class_means = np.array(class_means, dtype=np.float64)
    class_vars = np.array(class_vars, dtype=np.float64)
    class_ns = np.array(class_ns, dtype=np.float64)

    global_mean = np.sum(class_ns * class_means) / np.sum(class_ns)
    between = np.sum(class_ns * (class_means - global_mean) ** 2)
    within = np.sum(class_ns * class_vars)

    score = between / (within + 1e-12)
    return float(score), class_means, class_vars, class_ns

semantic_rows = []

for frame in multi_frame_data:
    fid = frame["frame_id"]
    label_path = LABELS_DIR / f"{fid}.label"
    semantic_labels = load_semantickitti_labels(label_path)

    valid_classes = get_valid_classes_notebook02_style(
        semantic_labels,
        min_points_per_class=500,
        ignored_classes=(0, 1, 255)
    )

    score_intensity, _, _, ns = fisher_separability(
        frame["intensity"],
        semantic_labels,
        valid_classes
    )

    score_pr, _, _, _ = fisher_separability(
```

```
        frame["pseudo_reflectivity"],
        semantic_labels,
        valid_classes
    )

    semantic_rows.append({
        "frame_id": fid,
        "num_valid_classes": len(valid_classes),
        "valid_classes": valid_classes,
        "intensity_sep_score": score_intensity,
        "pr_sep_score": score_pr,
        "absolute_gain": score_pr - score_intensity,
        "relative_gain": (score_pr - score_intensity) /
(score_intensity + 1e-12),
    })

semantic_df = pd.DataFrame(semantic_rows)

print("Frame 000000 sanity check")
row0 = semantic_df.iloc[0]
print(" - valid classes       :", row0["valid_classes"])
print(" - num valid classes   :", row0["num_valid_classes"])
print(" - intensity score     :",
f'{row0["intensity_sep_score"]:.6f}')
print(" - pseudo-reflectivity :", f'{row0["pr_sep_score"]:.6f}')
print(" - absolute gain       :", f'{row0["absolute_gain"]:.6f}')
print(" - relative gain       :", f'{row0["relative_gain"]:.6f}')
print()

print("First 10 frame-wise results")
print(semantic_df[[
    "frame_id",
    "num_valid_classes",
    "intensity_sep_score",
    "pr_sep_score",
    "absolute_gain",
    "relative_gain"
]].head(10))
print()

print("30-frame semantic separability summary")
print(" - intensity score mean        :",
semantic_df["intensity_sep_score"].mean())
print(" - pseudo-reflectivity mean    :",
semantic_df["pr_sep_score"].mean())
print(" - absolute gain mean          :",
semantic_df["absolute_gain"].mean())
print(" - relative gain mean          :",
semantic_df["relative_gain"].mean())
```

```
Frame 000000 sanity check
 - valid classes       : [10, 40, 44, 48, 50, 52, 60, 70, 71, 72, 80,
99]
 - num valid classes   : 12
 - intensity score     : 0.381428
 - pseudo-reflectivity : 0.577238
 - absolute gain       : 0.195809
 - relative gain       : 0.513358

First 10 frame-wise results
  frame_id  num_valid_classes  intensity_sep_score  pr_sep_score  \
0   000000                 12             0.381428      0.577238
1   000001                 12             0.387196      0.562421
2   000002                 12             0.401164      0.535149
3   000003                 12             0.419656      0.520715
4   000004                 12             0.423778      0.505296
5   000005                 13             0.454864      0.512665
6   000006                 13             0.470120      0.514486
7   000007                 12             0.492163      0.513640
8   000008                 12             0.523754      0.498392
9   000009                 12             0.547563      0.495476

   absolute_gain  relative_gain
0       0.195809       0.513358
1       0.175224       0.452547
2       0.133986       0.333993
3       0.101058       0.240812
4       0.081518       0.192361
5       0.057801       0.127073
6       0.044366       0.094372
7       0.021477       0.043638
8      -0.025361      -0.048422
9      -0.052087      -0.095125

30-frame semantic separability summary
 - intensity score mean        : 0.4504474749539853
 - pseudo-reflectivity mean    : 0.4752031096261892
 - absolute gain mean          : 0.024755634672203822
 - relative gain mean          : 0.09162314535748868
```

Using the same Fisher-style separability metric and class-filtering logic as Notebook 02, the frame-wise semantic comparison is now directly consistent with the earlier single-frame analysis.

Across the 30-frame motion window:

- pseudo-reflectivity is stronger than raw intensity in the early frames,
- its advantage gradually decreases,
- and the ordering flips in part of the sequence.

Overall, pseudo-reflectivity still achieves a slightly higher mean separability score across the full window, but the advantage is clearly motion- and scene-dependent rather than uniform.

Step 11: Visualize temporal semantic separability across motion

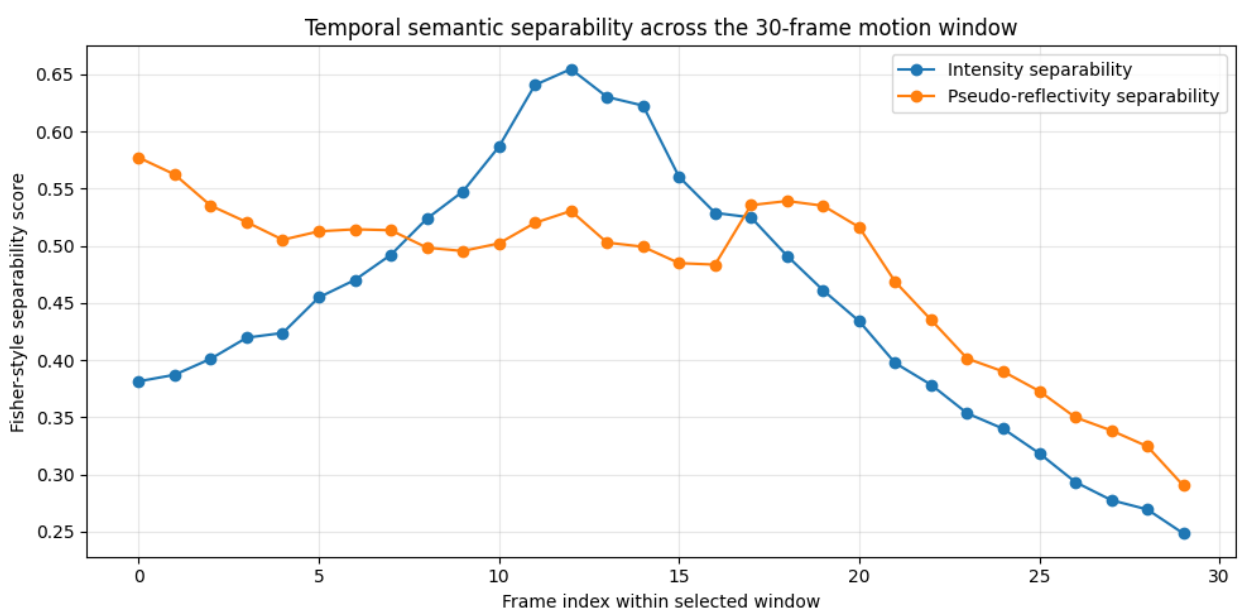
Now we plot the frame-wise semantic separability scores over the 30-frame window.

This shows how the semantic discriminative strength of:

- raw intensity.
- pseudo-reflectivity.

changes through motion when evaluated with the same metric used in Notebook 02.

```
frame_idx = np.arange(len(semantic_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, semantic_df["intensity_sep_score"], marker="o",
label="Intensity separability")
plt.plot(frame_idx, semantic_df["pr_sep_score"], marker="o",
label="Pseudo-reflectivity separability")

plt.xlabel("Frame index within selected window")
plt.ylabel("Fisher-style separability score")
plt.title("Temporal semantic separability across the 30-frame motion
window")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



The temporal semantic separability plot shows that the relative usefulness of raw intensity and pseudo-reflectivity changes across motion.

- pseudo-reflectivity is stronger in the early part of the sequence,
- raw intensity becomes more separable in the middle region,
- pseudo-reflectivity regains the lead later, although both signals decline toward the end.

This indicates that semantic discriminative strength is strongly scene-dependent over time, but pseudo-reflectivity still retains a slight overall advantage across the full 30-frame window.

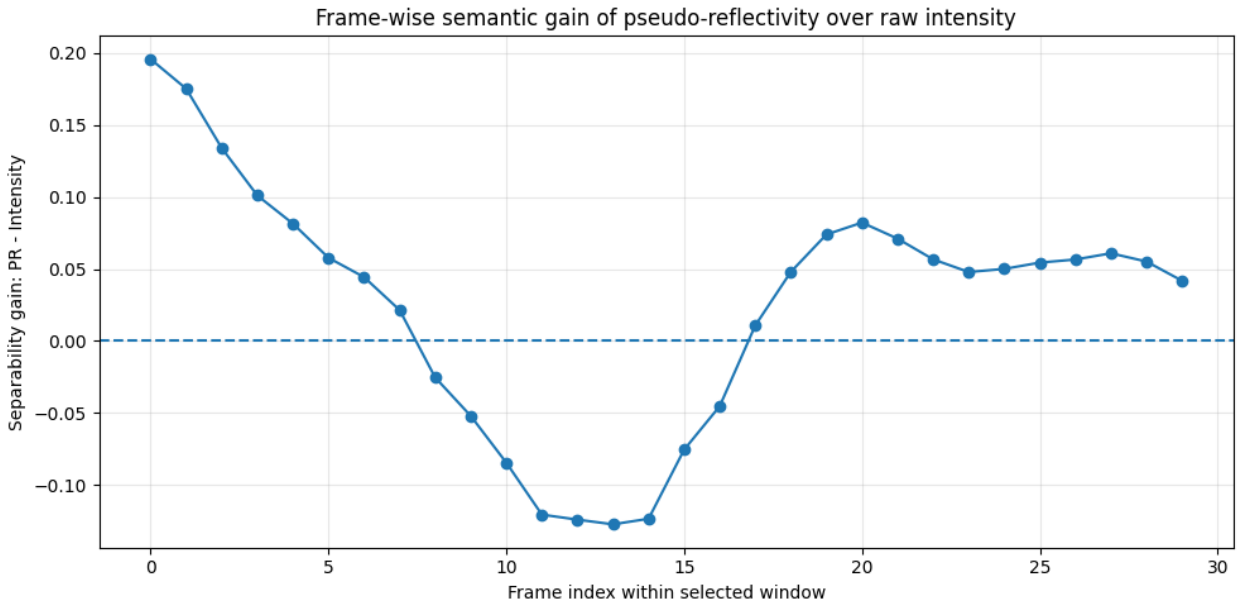
Step 12: Plot the separability gain of pseudo-reflectivity over raw intensity

To make the comparison more explicit, we now plot the frame-wise gain:

$$\Delta_{\text{sep}} = \text{Sep}(\hat{\rho}) - \text{Sep}(I)$$

Positive values mean pseudo-reflectivity is more semantically separable than raw intensity in that frame, while negative values mean the opposite.

```
frame_idx = np.arange(len(semantic_df))

plt.figure(figsize=(10, 5))
plt.plot(frame_idx, semantic_df["absolute_gain"], marker="o")
plt.axhline(0.0, linestyle="--")

plt.xlabel("Frame index within selected window")
plt.ylabel("Separability gain: PR - Intensity")
plt.title("Frame-wise semantic gain of pseudo-reflectivity over raw
intensity")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Brief observation

The frame-wise gain plot makes the temporal comparison explicit.

- pseudo-reflectivity provides a strong semantic advantage in the early frames,
- this advantage reverses in the middle of the sequence,
- and then becomes positive again in the later frames.

This confirms that the benefit of pseudo-reflectivity is not uniform through motion, but varies with the evolving scene content and geometry.

Step 13: Identify the strongest and weakest frames for pseudo-reflectivity gain

Now we summarize where pseudo-reflectivity helps the most and where it hurts the most.

This gives us concrete anchor frames for later visual inspection, interpretation, and possible GIF-based comparison.

```
top_k = 5

best_gain_frames = semantic_df.nlargest(top_k, "absolute_gain")[
    ["frame_id", "intensity_sep_score", "pr_sep_score",
"absolute_gain", "relative_gain"]
].reset_index(drop=True)

worst_gain_frames = semantic_df.nsmallest(top_k, "absolute_gain")[
    ["frame_id", "intensity_sep_score", "pr_sep_score",
"absolute_gain", "relative_gain"]
].reset_index(drop=True)
```

```
print("Top 5 frames where pseudo-reflectivity helps the most")
print(best_gain_frames)
print()
print("Top 5 frames where pseudo-reflectivity helps the least / hurts
the most")
print(worst_gain_frames)
```

```
Top 5 frames where pseudo-reflectivity helps the most
  frame_id  intensity_sep_score  pr_sep_score  absolute_gain
relative_gain
0   000000             0.381428      0.577238       0.195809
0.513358
1   000001             0.387196      0.562421       0.175224
0.452547
2   000002             0.401164      0.535149       0.133986
0.333993
3   000003             0.419656      0.520715       0.101058
0.240812
4   000020             0.434072      0.516331       0.082260
0.189507

Top 5 frames where pseudo-reflectivity helps the least / hurts the
most
  frame_id  intensity_sep_score  pr_sep_score  absolute_gain
relative_gain
0   000013             0.630274      0.502886      -0.127387      -
0.202115
1   000012             0.654662      0.530535      -0.124127      -
0.189604
2   000014             0.622705      0.499203      -0.123502      -
0.198331
3   000011             0.640741      0.520092      -0.120650      -
0.188297
4   000010             0.586710      0.502133      -0.084577      -
0.144154
```

Brief observation

The strongest positive pseudo-reflectivity gains occur mainly at the beginning of the 30-frame window, while the strongest negative gains are concentrated in the middle portion of the sequence.

This localizes the temporal transition observed earlier:

- early frames favor pseudo-reflectivity,
- middle frames favor raw intensity,
- later frames return to a milder pseudo-reflectivity advantage.

These identified frames provide useful anchors for later qualitative visualization and GIF-based inspection.

Step 14: Print a compact final quantitative summary for Notebook 03

Now we collect the main temporal findings of this notebook into one compact summary.

This includes:

- average range dependence over the 30-frame window,
- average semantic separability over the 30-frame window,
- mean gain of pseudo-reflectivity over raw intensity,
- and the best and worst gain frames for later visual inspection.

```
best_row = semantic_df.loc[semantic_df["absolute_gain"].idxmax()]
worst_row = semantic_df.loc[semantic_df["absolute_gain"].idxmin()]

print("NOTEBOOK 03: FINAL TEMPORAL SUMMARY")
print()

print("[Range Dependence Across 30 Frames]")
print(f"mean corr(raw intensity, range)    =
{corr_df['corr_intensity_range'].mean():.6f}")
print(f"mean corr(I * R, range)            =
{corr_df['corr_pr_range'].mean():.6f}")
print()

print("[Semantic Separability Across 30 Frames]")
print(f"mean raw intensity score           =
{semantic_df['intensity_sep_score'].mean():.6f}")
print(f"mean pseudo-reflectivity score     =
{semantic_df['pr_sep_score'].mean():.6f}")
print(f"mean absolute gain                 =
{semantic_df['absolute_gain'].mean():.6f}")
print(f"mean relative gain                 =
{semantic_df['relative_gain'].mean():.6f}")
print()

print("[Best Frame for Pseudo-Reflectivity Gain]")
print(f"frame_id                           = {best_row['frame_id']}")
print(f"intensity score                    =
{best_row['intensity_sep_score']:.6f}")
print(f"pseudo-reflectivity score          =
{best_row['pr_sep_score']:.6f}")
print(f"absolute gain                      =
{best_row['absolute_gain']:.6f}")
print(f"relative gain                      =
{best_row['relative_gain']:.6f}")
print()

print("[Worst Frame for Pseudo-Reflectivity Gain]")
print(f"frame_id                           = {worst_row['frame_id']}")
print(f"intensity score                    =
```

```
{worst_row['intensity_sep_score']:.6f}")
print(f"pseudo-reflectivity score          =
{worst_row['pr_sep_score']:.6f}")
print(f"absolute gain                      =
{worst_row['absolute_gain']:.6f}")
print(f"relative gain                      =
{worst_row['relative_gain']:.6f}")
```

```
NOTEBOOK 03: FINAL TEMPORAL SUMMARY

[Range Dependence Across 30 Frames]
mean corr(raw intensity, range)    = -0.206539
mean corr(I * R, range)            = 0.364621

[Semantic Separability Across 30 Frames]
mean raw intensity score           = 0.450447
mean pseudo-reflectivity score     = 0.475203
mean absolute gain                 = 0.024756
mean relative gain                 = 0.091623

[Best Frame for Pseudo-Reflectivity Gain]
frame_id                           = 000000
intensity score                    = 0.381428
pseudo-reflectivity score          = 0.577238
absolute gain                      = 0.195809
relative gain                      = 0.513358

[Worst Frame for Pseudo-Reflectivity Gain]
frame_id                           = 000013
intensity score                    = 0.630274
pseudo-reflectivity score          = 0.502886
absolute gain                      = -0.127387
relative gain                      = -0.202115
```

Visualizations (GIF)

Compute fixed visualization ranges for the full 30-frame window

Before rendering any GIF frames, we compute global visualization limits across all selected frames.

We do this so that:

- BEV views use the same spatial window throughout the sequence,
- intensity uses one fixed color scale across all 30 frames,
- pseudo-reflectivity uses one fixed color scale across all 30 frames.

This prevents frame-by-frame autoscaling from making the GIF visually misleading.

```
all_xyz = np.vstack([frame["xyz"] for frame in multi_frame_data])
all_intensity = np.concatenate([frame["intensity"] for frame in
multi_frame_data])
all_pr = np.concatenate([frame["pseudo_reflectivity"] for frame in
multi_frame_data])

# Global spatial limits for BEV
x_min, x_max = float(np.min(all_xyz[:, 0])), float(np.max(all_xyz[:,
0]))
y_min, y_max = float(np.min(all_xyz[:, 1])), float(np.max(all_xyz[:,
1]))

# Use robust quantile clipping for color stability
intensity_vmin = float(np.percentile(all_intensity, 1))
intensity_vmax = float(np.percentile(all_intensity, 99))

pr_vmin = float(np.percentile(all_pr, 1))
pr_vmax = float(np.percentile(all_pr, 99))

print("Global BEV limits")
print(f" - x range: [{x_min:.3f}, {x_max:.3f}]")
print(f" - y range: [{y_min:.3f}, {y_max:.3f}]")
print()

print("Global intensity color limits")
print(f" - vmin: {intensity_vmin:.6f}")
print(f" - vmax: {intensity_vmax:.6f}")
print()

print("Global pseudo-reflectivity color limits")
print(f" - vmin: {pr_vmin:.6f}")
print(f" - vmax: {pr_vmax:.6f}")
```

```
Global BEV limits
 - x range: [-80.393, 79.885]
 - y range: [-72.722, 78.965]

Global intensity color limits
 - vmin: 0.000000
 - vmax: 0.710000

Global pseudo-reflectivity color limits
 - vmin: 0.000000
 - vmax: 12.038756
```

Define a fixed forward-view window for the FOV panels

Now we define the visualization window for the sensor-centric forward view.

This window should remain fixed across all 30 frames so that the FOV panels are directly comparable over time.

```
# Global sensor-frame limits for a forward-view style plot
# x = forward, y = left-right, z = height

all_x = all_xyz[:, 0]
all_y = all_xyz[:, 1]
all_z = all_xyz[:, 2]

# Keep only forward-facing points to define a practical FOV window
forward_mask = all_x > 0.0

fov_x_min = 0.0
fov_x_max = float(np.percentile(all_x[forward_mask], 99))

fov_y_abs = float(np.percentile(np.abs(all_y[forward_mask]), 99))
fov_y_min = -fov_y_abs
fov_y_max = fov_y_abs

fov_z_min = float(np.percentile(all_z[forward_mask], 1))
fov_z_max = float(np.percentile(all_z[forward_mask], 99))

print("Fixed FOV limits")
print(f" - x (forward) : [{fov_x_min:.3f}, {fov_x_max:.3f}]")
print(f" - y (lateral) : [{fov_y_min:.3f}, {fov_y_max:.3f}]")
print(f" - z (height)  : [{fov_z_min:.3f}, {fov_z_max:.3f}]")
```

```
Fixed FOV limits
 - x (forward) : [0.000, 44.437]
 - y (lateral) : [-24.467, 24.467]
 - z (height)  : [-1.976, 0.957]
```

Render one test comparison frame

Before creating the full GIF, we render a single comparison board for one frame.

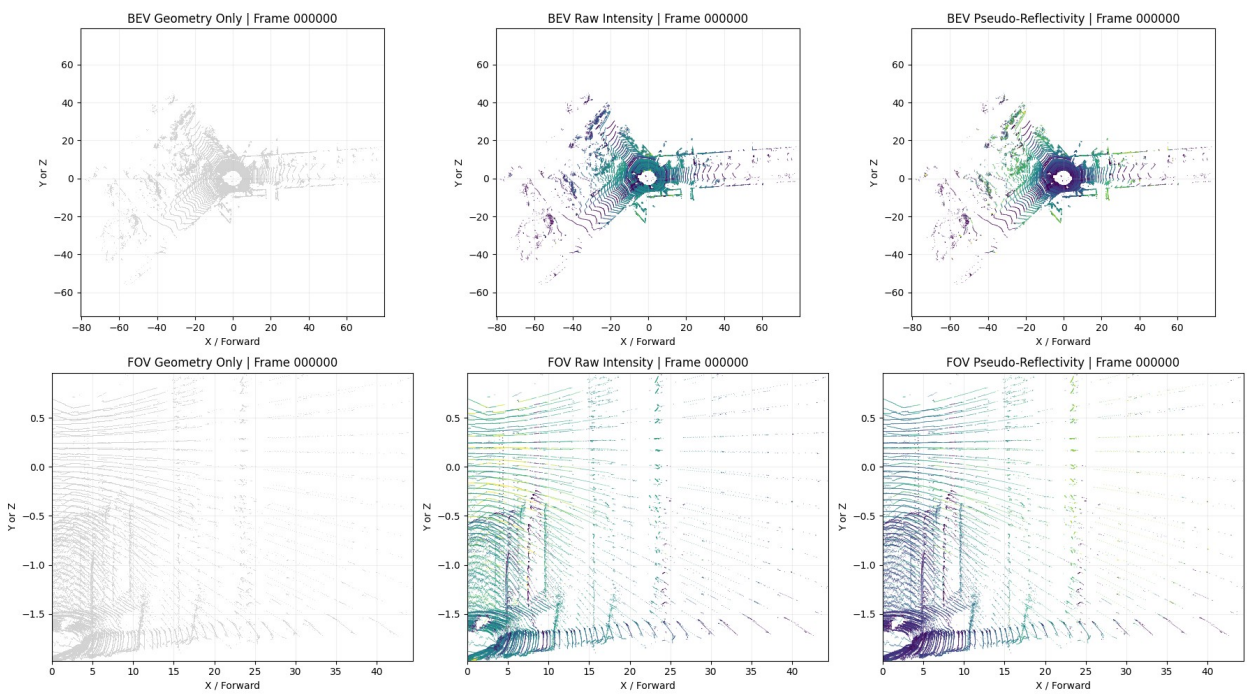
This lets us verify that:

- the 2×3 layout is correct,
- BEV and FOV views are readable,
- color scales behave properly,
- and the neutral, intensity, and pseudo-reflectivity panels are visually meaningful.

```
import matplotlib.pyplot as plt

test_idx = 0
frame = multi_frame_data[test_idx]

xyz = frame["xyz"]
intensity = frame["intensity"]
pr = frame["pseudo_reflectivity"]
fid = frame["frame_id"]
```

```python
x = xyz[:, 0]
y = xyz[:, 1]
z = xyz[:, 2]

# Forward-facing mask for FOV panels
fwd = x > 0.0

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Top row: BEV

axes[0, 0].scatter(x, y, s=0.3, c="lightgray", linewidths=0)
axes[0, 0].set_title(f"BEV Geometry Only | Frame {fid}")
axes[0, 0].set_xlim(x_min, x_max)
axes[0, 0].set_ylim(y_min, y_max)
axes[0, 0].set_aspect("equal")

sc1 = axes[0, 1].scatter(
    x, y, s=0.3, c=intensity, cmap="viridis",
    vmin=intensity_vmin, vmax=intensity_vmax, linewidths=0
)
axes[0, 1].set_title(f"BEV Raw Intensity | Frame {fid}")
axes[0, 1].set_xlim(x_min, x_max)
axes[0, 1].set_ylim(y_min, y_max)
axes[0, 1].set_aspect("equal")

sc2 = axes[0, 2].scatter(
    x, y, s=0.3, c=pr, cmap="viridis",
    vmin=pr_vmin, vmax=pr_vmax, linewidths=0
)
axes[0, 2].set_title(f"BEV Pseudo-Reflectivity | Frame {fid}")
axes[0, 2].set_xlim(x_min, x_max)
axes[0, 2].set_ylim(y_min, y_max)
axes[0, 2].set_aspect("equal")

# Bottom row: FOV
# x = forward, z = height

axes[1, 0].scatter(x[fwd], z[fwd], s=0.3, c="lightgray", linewidths=0)
axes[1, 0].set_title(f"FOV Geometry Only | Frame {fid}")
axes[1, 0].set_xlim(fov_x_min, fov_x_max)
axes[1, 0].set_ylim(fov_z_min, fov_z_max)

axes[1, 1].scatter(
    x[fwd], z[fwd], s=0.3, c=intensity[fwd], cmap="viridis",
    vmin=intensity_vmin, vmax=intensity_vmax, linewidths=0
)
axes[1, 1].set_title(f"FOV Raw Intensity | Frame {fid}")
axes[1, 1].set_xlim(fov_x_min, fov_x_max)
```

```
axes[1, 1].set_ylim(fov_z_min, fov_z_max)

axes[1, 2].scatter(
    x[fwd], z[fwd], s=0.3, c=pr[fwd], cmap="viridis",
    vmin=pr_vmin, vmax=pr_vmax, linewidths=0
)
axes[1, 2].set_title(f"FOV Pseudo-Reflectivity | Frame {fid}")
axes[1, 2].set_xlim(fov_x_min, fov_x_max)
axes[1, 2].set_ylim(fov_z_min, fov_z_max)

for ax in axes.ravel():
    ax.grid(True, alpha=0.2)
    ax.set_xlabel("X / Forward")
    ax.set_ylabel("Y or Z")

plt.tight_layout()
plt.show()
```



Render all 30 comparison frames and assemble the GIF

Now we generate the full 30-frame comparison GIF.

Each GIF frame contains a 2×3 layout:

- top row: BEV geometry, BEV intensity, BEV pseudo-reflectivity.
- bottom row: forward-view geometry, forward-view intensity, forward-view pseudo-reflectivity.

All panels use fixed spatial limits and fixed color normalization so that temporal changes remain visually comparable across the sequence.

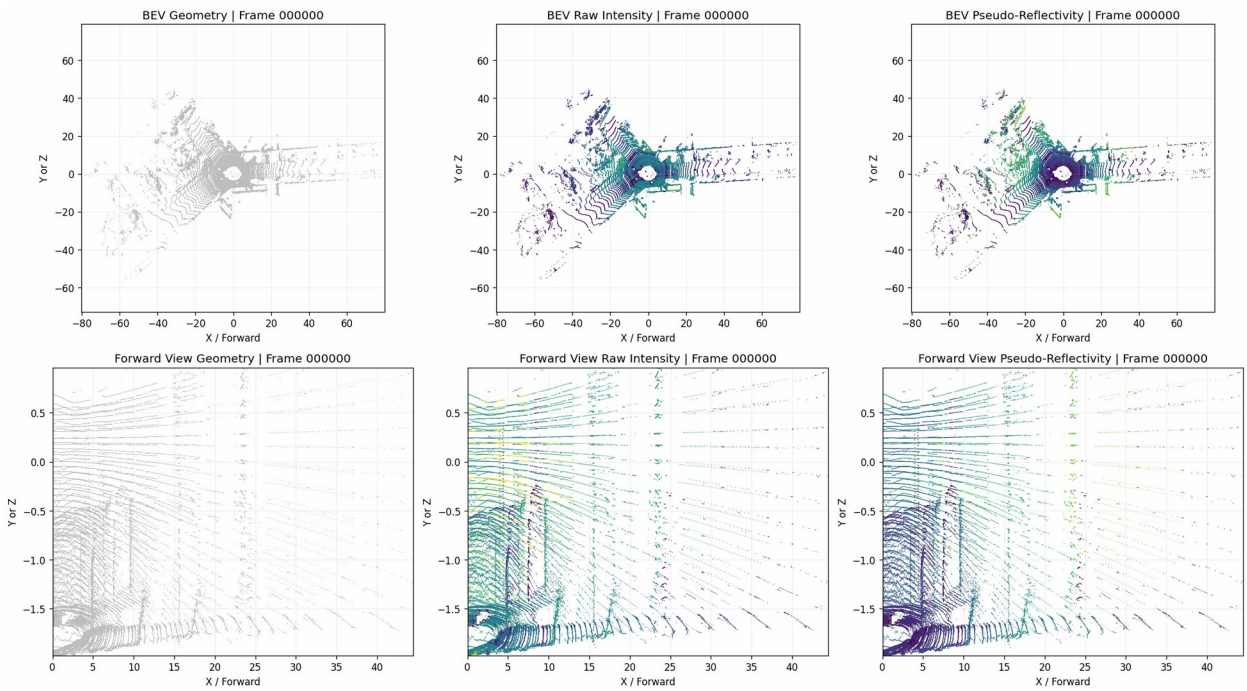
```
from pathlib import Path
import imageio.v2 as imageio

gif_dir = Path("artifacts")
gif_dir.mkdir(exist_ok=True)

temp_dir = gif_dir / "gif_frames_notebook03"
temp_dir.mkdir(exist_ok=True)

gif_path = gif_dir / "notebook03_bev_forward_intensity_vs_pr_30f.gif"

point_size = 0.5
geom_color = "#bfbfbf"
fps = 3

rendered_frame_paths = []

for idx, frame in enumerate(multi_frame_data):
    xyz = frame["xyz"]
    intensity = frame["intensity"]
    pr = frame["pseudo_reflectivity"]
    fid = frame["frame_id"]

    x = xyz[:, 0]
    y = xyz[:, 1]
    z = xyz[:, 2]

    fwd = x > 0.0

    fig, axes = plt.subplots(2, 3, figsize=(18, 10))

    # Top row: BEV

    axes[0, 0].scatter(x, y, s=point_size, c=geom_color, linewidths=0)
    axes[0, 0].set_title(f"BEV Geometry | Frame {fid}")
    axes[0, 0].set_xlim(x_min, x_max)
    axes[0, 0].set_ylim(y_min, y_max)
    axes[0, 0].set_aspect("equal")

    axes[0, 1].scatter(
        x, y, s=point_size, c=intensity, cmap="viridis",
        vmin=intensity_vmin, vmax=intensity_vmax, linewidths=0
    )
    axes[0, 1].set_title(f"BEV Raw Intensity | Frame {fid}")
    axes[0, 1].set_xlim(x_min, x_max)
    axes[0, 1].set_ylim(y_min, y_max)
    axes[0, 1].set_aspect("equal")
```

```
    axes[0, 2].scatter(
        x, y, s=point_size, c=pr, cmap="viridis",
        vmin=pr_vmin, vmax=pr_vmax, linewidths=0
    )
    axes[0, 2].set_title(f"BEV Pseudo-Reflectivity | Frame {fid}")
    axes[0, 2].set_xlim(x_min, x_max)
    axes[0, 2].set_ylim(y_min, y_max)
    axes[0, 2].set_aspect("equal")

    # Bottom row: Forward view

    axes[1, 0].scatter(x[fwd], z[fwd], s=point_size, c=geom_color,
linewidths=0)
    axes[1, 0].set_title(f"Forward View Geometry | Frame {fid}")
    axes[1, 0].set_xlim(fov_x_min, fov_x_max)
    axes[1, 0].set_ylim(fov_z_min, fov_z_max)

    axes[1, 1].scatter(
        x[fwd], z[fwd], s=point_size, c=intensity[fwd],
cmap="viridis",
        vmin=intensity_vmin, vmax=intensity_vmax, linewidths=0
    )
    axes[1, 1].set_title(f"Forward View Raw Intensity | Frame {fid}")
    axes[1, 1].set_xlim(fov_x_min, fov_x_max)
    axes[1, 1].set_ylim(fov_z_min, fov_z_max)

    axes[1, 2].scatter(
        x[fwd], z[fwd], s=point_size, c=pr[fwd], cmap="viridis",
        vmin=pr_vmin, vmax=pr_vmax, linewidths=0
    )
    axes[1, 2].set_title(f"Forward View Pseudo-Reflectivity | Frame
{fid}")
    axes[1, 2].set_xlim(fov_x_min, fov_x_max)
    axes[1, 2].set_ylim(fov_z_min, fov_z_max)

    for ax in axes.ravel():
        ax.grid(True, alpha=0.2)
        ax.set_xlabel("X / Forward")
        ax.set_ylabel("Y or Z")

    plt.tight_layout()

    frame_png = temp_dir / f"frame_{idx:03d}.png"
    plt.savefig(frame_png, dpi=120, bbox_inches="tight")
    plt.close(fig)

    rendered_frame_paths.append(frame_png)

images = [imageio.imread(p) for p in rendered_frame_paths]
imageio.mimsave(gif_path, images, fps=fps)
```

```
print("GIF created at:", gif_path.resolve())
print("Total frames:", len(rendered_frame_paths))
print("Approx duration (s):", len(rendered_frame_paths) / fps)
```

```
GIF created at:
/home/twilightpriest/GitHub/reflect-aug-seg/notebooks/artifacts/notebook03_bev_forward_intensity_vs_pr_30f.gif
Total frames: 30
Approx duration (s): 10.0
```

```
from IPython.display import Image, display
display(Image(filename=str(gif_path)))
```



Qualitative Observation

The 30-frame comparison GIF provides a temporal visual check of the reflectivity-style signal in both bird's-eye and forward views.

Several patterns are visible:

- the spatial structure of the scene remains consistent across time,
- raw intensity and pseudo-reflectivity produce visibly different highlight patterns,
- pseudo-reflectivity tends to emphasize some stronger returns differently from raw intensity,
- and these differences evolve smoothly rather than chaotically across the motion window.

This supports the quantitative findings of Notebook 03: the pseudo-reflectivity proxy is temporally stable and visually structured, but its usefulness remains scene-dependent rather than uniformly dominant.

Final Conclusion

Notebook 03 extended the analysis from a single LiDAR frame to a short 30-frame motion window and tested whether the pseudo-reflectivity proxy remains meaningful through time.

The main findings are:

- the pseudo-reflectivity signal remains numerically stable across motion,
- its distribution evolves smoothly rather than erratically,
- it stays clearly range-dependent, so it should not be interpreted as geometry-invariant reflectivity,
- and it provides a small average semantic separability advantage over raw intensity across the selected 30-frame sequence.

At the same time, the gain is not uniform. Some parts of the motion window favor pseudo-reflectivity, while others favor raw intensity. This means the usefulness of the proxy depends on the evolving scene composition and geometry rather than acting as a universally superior replacement.

Taken together, the quantitative plots and the qualitative GIF support the same conclusion:

> the pseudo-reflectivity proxy is a temporally stable, visually structured, and sometimes semantically helpful range-aware feature, but its benefit is motion- and scene-dependent.

This makes it a defensible candidate for reflectivity-aware perception analysis in later stages of the project, while also setting clear limits on what can and cannot be claimed from the current formulation.